

## **COP 3515 – Advanced Program Design**

### **Semester Programming Project**

#### **Client Change Request (CCR-004)**

#### **Project: Student Information Management System (SIMS)**

**Customer:** University Registrar's Office

**Priority:** High

**Assigned To:** Student Developer

**Sprint:** Week 4

**Due Date:** Beginning of Lesson 10

---

#### **Business Background**

The University Registrar's Office has been very pleased with the progress of the Student Information Management System (SIMS). The application now records student information, tracks course grades, and determines each student's academic standing.

One limitation remains: when the program exits, all student information is lost. Advisors must re-enter the same information every time they use the application. This is inefficient and increases the possibility of data entry errors.

The Registrar would like the application to permanently save student information so it can be retrieved later. Employees should also be able to verify that the information stored on disk matches what was originally entered into the system.

This enhancement should be added to the existing SIMS application developed during **CCR-001, CCR-002, and CCR-003. Do not create a new program.** Continue enhancing your existing codebase.

---

## Business Requirements

The Registrar would like the application to continue performing **all functionality from CCR-001 through CCR-003**.

In addition, the software should:

1. Save the student's information to a text file.
2. Save the student's five course grades.
3. Save the student's GPA.
4. Save the student's academic standing.
5. Read the information back from the text file.
6. Display the information that was read from the file.
7. Allow the employee to verify that the information read from the file matches the information that was originally entered.

For this release, the application only needs to store information for **one student**.

---

## Acceptance Criteria

The project will be considered complete when all of the following requirements have been met.

- ☐ The program compiles without errors or warnings.
- ☐ All functionality from **CCR-001** continues to work correctly.
- ☐ All functionality from **CCR-002** continues to work correctly.
- ☐ All functionality from **CCR-003** continues to work correctly.
- ☐ Student information is successfully written to a text file.
- ☐ Course grades are successfully written to the file.
- ☐ GPA and academic standing are written to the file.
- ☐ The program successfully reads the file.
- ☐ The information read from the file matches the information originally entered.
- ☐ The recovered student report is neatly formatted.

☐ Program handles file errors gracefully.

---

### Technical Constraints

For this week's assignment, you may **only** use C language features covered through **Lesson 10**.

Permitted concepts include:

- Variables
- Constants
- Arrays
- Enumerations
- Mathematical operators
- Input/Output
- Selection statements
- File processing
- Character processing
- Type casting
- Formatted input/output (`printf`, `scanf` or similar)

Do **not** use language features that have not yet been introduced in class.

---

### Out of Scope

The Registrar is **not** requesting any of the following features at this time.

- Multiple students
- Menus
- Searching student records
- Editing saved records
- Binary files

- Encryption
- Databases
- Dynamic memory allocation
- Multiple data files

These features will be requested in future Client Change Requests.

---

## Questions for the Customer

Before writing any code, identify questions you would ask the customer to clarify the requirements.

Examples include:

1. What should the name of the data file be?
  2. Should the program create the file if it does not already exist?
  3. Should an existing file be overwritten or appended to?
  4. What should happen if the file cannot be opened?
  5. Should the program automatically read the file after saving it?
  6. Should the student report displayed after reading the file exactly match the original report?
  7. Should course names also be saved, or only course grades?
  8. Should the academic standing be recalculated after reading the file, or simply read from the file?
  9. What should happen if the file contains invalid or incomplete data?
  10. Should blank lines be included in the text file to improve readability?
  11. Will future versions store multiple student records in the same file?
  12. Should employees be able to specify the filename in future versions?
-

## Example Program Execution

### Example 1

#### User Input

Student ID: 1234567

Student Name: Alice Johnson

Current GPA: 3.84

#### Course Grades

95

88

91

84

100

#### Program Output

Student information successfully saved.

Reading student information...

File successfully loaded.

Recovered Student Record

-----

# Student Information Management System

Version 4.0

-----

Student ID : 1234567

Student Name : Alice Johnson

Current GPA : 3.84

Academic Standing : Honors

## Course Grades

95

88

91

84

100

Average Grade : 91.60

Highest Grade : 100

Lowest Grade : 84

---

## Example 2

### Expected File Contents

1234567

Alice Johnson

3.84

Honors

95

88

91

84

100

---

### Example 3

#### File Error

the data file cannot be opened:

ERROR

Unable to open student\_records.txt

Please verify that the file exists and that you have permission to access it.

---

#### Regression Testing Requirements

Since the Student Information Management System is developed incrementally, **every previously implemented feature must continue to operate correctly.**

Your submission must demonstrate that:

- CCR-001 functionality still works.
- CCR-002 functionality still works.
- CCR-003 functionality still works.
- CCR-004 functionality works correctly.

At a minimum, include the following regression tests.

| Test Description                      | Expected Result                     |
|---------------------------------------|-------------------------------------|
| 1 Enter valid student information     | Student report displayed correctly  |
| 2 Save information to file            | File created successfully           |
| 3 Read information from file          | Data matches original input         |
| 4 Compare original and recovered data | Information is identical            |
| 5 Attempt to read a missing file      | Appropriate error message displayed |

For **each** regression test include:

- Test description
- Input
- Expected output
- Actual output
- Pass/Fail result

If a defect is discovered:

1. Document the defect.
  2. Correct the program.
  3. Repeat the affected test.
  4. Verify that previously implemented functionality still works correctly.
-

## AI Usage Policy

Artificial Intelligence tools (such as ChatGPT, GitHub Copilot, Claude, or Gemini) may be used during this assignment.

However, you remain responsible for:

- Understanding every line of submitted code.
- Thoroughly testing all functionality.
- Correcting any defects.
- Demonstrating that both new and existing features operate correctly.

AI-generated code that does not function correctly will not receive full credit.

---

## Submission Requirements

Submit a single ZIP file containing:

- main.c
- Any additional source files (if applicable)
- The generated data file (student\_records.txt) created by your program

Also include a PDF containing:

- Screenshots demonstrating successful program execution
  - Screenshots showing the data file contents
  - Results of all required regression tests
  - A brief description of any bugs encountered and how they were corrected
  - Answers to the **Questions for the Customer**
  - Any assumptions made because the customer requirements were ambiguous
-

## Looking Ahead

This is the **fourth** release of the Student Information Management System (SIMS).

Future Client Change Requests will continue expanding the system by introducing loops, functions, pointers, dynamic memory allocation, and additional software engineering concepts. Eventually, the system will manage multiple students and provide a complete command-line student information system.

Remember that **professional software evolves over time**. New features should be added without breaking existing functionality. Each release should leave the application more capable while maintaining the reliability established by previous releases through comprehensive regression testing.